External Software Documentation

- **End-user documentation**: gives end users basic instructions on how to use, install and troubleshoot the software (e.g., user guides, tutorials).
- **Enterprise user documentation**: used for IT staff who deploy the software across the enterprise.
- **Just-in-time documentation**: provides end users with support documentation at the exact time they will need it (e.g., FAQ pages, how-to documents)

Internal Software Documentation

- **Administrative documentation**: the high-level administrative guidelines, roadmaps and product requirements for the software development team and project managers working on the software (e.g., status reports, meeting notes).
- **Developer documentation**: provides instructions to developers for building the software and guides them through the development process.
  - Requirements documentation
  - Architecture & design documentation
  - Code Comments
  - API documentation
  - Readme, release notes, etc.

Self-documenting Code

有意义的命名和目录结构

- Self-documenting (or self-describing) code follow naming conventions and structured programming conventions that enable use and understanding of the system without prior specific knowledge
  - Meaningful directory structure
  - Meaningful file/class/method/variable names
- Main goal: allowing developers to understand code at a glance

```
float a, b, c; a=9.81; b=5; c= .5*a*(b^2);
```

vs

```
const float gravitationalForce = 9.81;
float timeInSeconds = 5;
float displacement = (1 / 2) * gravitationalForce * (timeInSeconds ^ 2);
```

Good code doesn't need documentation. It's self-explaining

Code Comments

- Sometimes when the code alone can't provide context or clarify intent, the developer may write extra descriptions, i.e., code comments.
- Code comments enhance readability. They facilitate code reviews, refactoring, and maintenance.
- Code comments are ignored by compilers and interpreters when producing the final executable. Thus, they incur no runtime performance overhead. However, too many or unnecessary comments reduce readability.



应解释目的与原因

- Comments should be used only to explain the **intent** behind code that cannot be refactored to explain itself
- Mostly used for providing **additional context**, instead of simply repeating the code
- Should answer **WHY**, instead of WHAT

```
const float a = 9.81; //gravitational force
float b = 5; //time in seconds
float c = (1/2)*a*(b^2) //multiply the time and gravity together to get displacement.
```

vs

```
/* compute displacement with Newton's equation x = v₀t + ½at² */
const float gravitationalForce = 9.81;
float timeInSeconds = 5;
float displacement = (1 / 2) * gravitationalForce * (timeInSeconds ^ 2);
```

```
1       while (! finished) {
2         try {
3           Thread.sleep(10); // so that program gets time to handle xxx logics
4         }
5         catch (InterruptedException e) {
…           ……
…         }
…       }
```

Meaningful code comment complements / explains the intention of the code

```
1   //WARNING: 本测试用例需要耗费大约15分钟时间，因此日常集成时不运行
2   @Ignore
3   @Test
4   public void testHighThroughput() {
5   //模拟十万个线程同时进行访问
6   }
```

Meaningful code comment gives necessary warnings

```
1   //Notice：下列数据库连接参数需要改为从外部配置文件中读取
2   String driver = "com.mysql.jdbc.Driver";
3   String url = "jdbc:mysql://localhost:3306/sqltestdb";
4   String user = "root";
5   String password = "123456";
6   Class.forName(driver);
7   con = DriverManager.getConnection(url,user,password);
```

Meaningful code comment reminds about self-admitted technical debt (自承认技术债): developers consciously perform the hack (suboptimal solution) due to time constraints or technical limitations

```
1  public int[] sortNumbers() {
2    //TODO: 需在这里补充真实的排序代码
3    return new int[]{1,2,3,4,5};
4  }
```

Meaningful code comment helps developers locate incomplete/unfinished implementations (//TODO cannot exist in delivered code)

– Examples

```
// make sure the port is greater or equal to 1024
if (port < 1024) {
  throw new InvalidPortError(port);
}
```

Bad
- No additional information
- WHAT

```
// port numbers below 1024 (the privileged or "well-known ports")
// require root access, which we don't have
if (port < 1024) {
  throw new InvalidPortError(port);
}
```

Okay
- Additional information
- WHY

```
if (!hasRootPrivileges(port)) {
  throw new InvalidPortError(port);
}

private boolean hasRootPrivileges(int port) {
  // port numbers below 1024 (the privileged or "well-known ports")
  // require root access on unix systems
  return port < 1024;
}
```

Better
- Refactored with meaningful name (`hasRootPrivileges`)

```
final static const HIGHEST_PRIVILEDGED_PORT = 1023;

private boolean hasRootPrivileges(int port) {
  // The privileged or "well-known ports" require root access on unix systems
  return port <= HIGHEST_PRIVILEDGED_PORT;
}
```

Even better
- Turn magic number into a constant with meaningful name
- Comment might no longer be needed

– Describing WHY

- Describe the design decisions to a class
- Describe the limitations of an implementation
- Describe usage assumptions of APIs
- Describe the purpose and intents of each file

Commenting Principles

- The best documentation is the code itself
- Make the code self-explainable and self-documenting
- Do not document bad code, refactor or rewrite it!
- WHY (rationales), not WHAT (implementations)

Javadoc

Javadoc (included in JDK) automatically generates API documentation from comments present in the Java source code.

- Javadoc comments structure look very similar to a regular multi-line comment, but the key difference is the extra asterisk at the beginning
- Javadoc style comments may contain HTML tags as well.

```
// This is a single line comment

/*
 * This is a regular multi-line comment
 */

/**
 * This is a Javadoc
 */
```

- Javadoc comments may be placed above any class, method, or field which we want to document.
- These comments are commonly made up of two sections:
  - The description of what we're commenting on
  - The standalone block tags (marked with the "@" symbol) which describe specific meta-data

Javadoc Tags

@author	A person who made significant contribution to the code. Applied only at the class, package, or overview level. Not included in Javadoc output. It's not recommended to include this tag since authorship changes often.
@param	A parameter that the method or constructor accepts. Write the description like this: @param count Sets the number of widgets you want included.
@deprecated	Lets users know the class or method is no longer used. This tag will be positioned in a prominent way in the Javadoc. Accompany it with a @see or {@link} tag as well.
@return	What the method returns.

@see	Creates a see also list. Use {@link} for the content to be linked.
{@link}	Used to create links to other classes or methods. Example: {@link Foo#bar} links to the method `bar` that belongs to the class `Foo`. To link to the method in the same class, just include #bar.
@since 2.0	The version since the feature was added.
@throws	The kind of exception the method throws. Note that your code must indicate an exception thrown in order for this tag to validate. Otherwise Javadoc will produce an error. @exception is an alternative tag.
@Override	performs a check to see if the method is an override. used with interfaces and abstract classes.

Class

```java
/**
* Hero is the main entity we'll be using to . . .
*
* Please see the {@link com.baeldung.javadoc.Person} class for true identity
* @author Captain America
*
*/
public class SuperHero extends Person {
    // fields and methods
}
```

Method

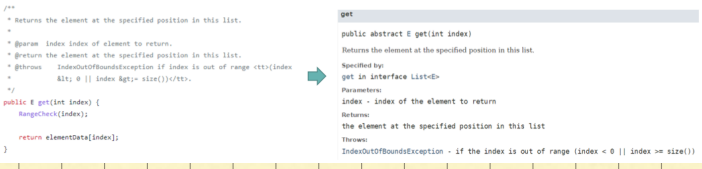
```java
/**
 * Returns the element at the specified position in this list.
 *
 * @param  index index of element to return.
 * @return the element at the specified position in this list.
 * @throws    IndexOutOfBoundsException if index is out of range <tt>(index
 *            &lt; 0 || index &gt;= size())</tt>.
 */
public E get(int index) {
    RangeCheck(index);

    return elementData[index];
}
```

java.util.ArrayList

Javadoc Generation

```
C:> javadoc -d C:\home\html -sourcepath C:\home\src java.my.package
```



get

public abstract E get(int index)

Returns the element at the specified position in this list.

Specified by:
get in interface List<E>

Parameters:
index - index of the element to return

Returns:
the element at the specified position in this list

Throws:
IndexOutOfBoundsException - if the index is out of range (index < 0 || index >= size())

Documentation Tools & Frameworks

- Python: Python has its built-in documentation tool called PyDoc, which generates HTML documentation from Python code.
- JavaScript: JSDoc is an open source API documentation generator for Javascript. It allows developers to document their code through comments.
- Sphinx: a documentation generator written and used by the Python community. https://www.sphinx-doc.org/en/master/
- Swagger: a popular framework for documenting REST APIs. https://swagger.io/docs/
- GitBook: a modern documentation platform where teams can document everything from products to internal knowledge bases and APIs. https://www.gitbook.com/

Tests

- Tests could be used to “document” how the code should behave
- Tests provide examples of how to use an API, with assertions
- Tests also give examples of edge case scenarios

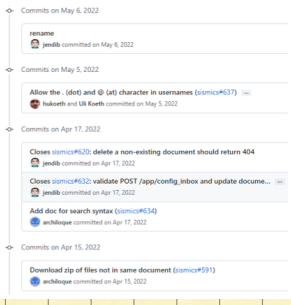
```
void shouldRetrieveValuesInOrderTheyAreAdded()
void shouldThrowExceptionIfStackIsEmpty()
void shouldThrowExceptionIfMaxThresholdIsReached()
```

Sample tests for Stack

Git Commit Message

GIT COMMIT MESSAGES

- Gives context of the change
- Explain “why this change is needed?” kinds of questions
- Commit messages live with the code with no clutter
- Always up-to-date w.r.t. the code changes



Commits on May 6, 2022

rename
jendib committed on May 6, 2022

Commits on May 5, 2022

Allow the . (dot) and @ (at) character in usernames (sismics#637)
hukoeth and Uli Koeth committed on May 5, 2022

Commits on Apr 17, 2022

Closes sismics#620: delete a non-existing document should return 404
jendib committed on Apr 17, 2022

Closes sismics#632: validate POST /app/config_inbox and update docume...
jendib committed on Apr 17, 2022

Add doc for search syntax (sismics#634)
archiloque committed on Apr 17, 2022

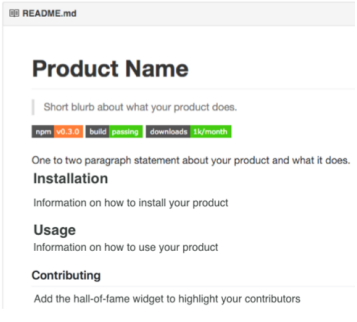
Commits on Apr 15, 2022

Download zip of files not in same document (sismics#591)
archiloque committed on Apr 15, 2022

README

README

- Provide high-level context such as
  - What’s this project is for?
  - Design choices
  - How to install
  - How to use
  - How to contribute
  - License
  - ……



README.md

Product Name

Short blurb about what your product does.

npm v0.3.0 build passing downloads 1k/month

One to two paragraph statement about your product and what it does.

Installation

Information on how to install your product

Usage

Information on how to use your product

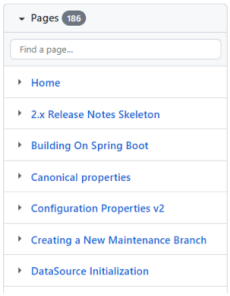
Contributing

Add the hall-of-fame widget to highlight your contributors

WIKIS

WIKIS

- Every repo on GitHub comes equipped with a section for hosting documentation, called a wiki.
- You can use wiki to share long-form content about your project, such as how to use it, how you designed it, or its core principles.
- A README file quickly tells what your project can do, while you can use a wiki to provide additional documentation.



https://github.com/spring-projects/spring-boot/wiki

Case Study: g3doc @ Google

Fragmentation is bad
- Hard to create docs
  - Context switching
  - Horrible tooling
- Hard to maintain docs
  - No explicit connection to code
  - No clear ownership
- Hard to find docs

Documentation will never be part of the engineering culture until it is integrated into the codebase and workflow

当文档集成入代码库与工作流时，就是工程文化的一部分

g3doc

Engineers find, create, and maintain docs using their regular workflow and developers tools

文档视作代码的一部分

```
Documentation is treated as code
```

DESIGN 1: DOCS LIVES WITH CODE

Docs are stored side-by-side with the source code in the codebase

DESIGN 2: USE MARKDOWN

Docs are written in markdown format (.md)

BENEFITS

- Docs are under version control like code
- Allow code review, diff, blame, fixes, and issue tracking
- g3doc renders beautiful html based on .md files
- Developers focus on the contents, instead of the presentation
- Docs have rich links to source code
- Easy for code search